

Adaptive Honeypot Controller

Project Report — AI Performance Analysis

Khrystyna Kodatska · ENGR-202C · UCLA

1. Project Description

The Adaptive Honeypot Controller is a Python/Streamlit application that simulates a network honeypot environment and deploys an autonomous AI agent to manage it in real time. The system tracks simulated attackers across six threat phases — Scanning, Brute Force, Exploit, Persistence, Data Exfiltration, and Dormant — and responds dynamically by adjusting login delays, deploying fake vulnerable endpoints, or planting honeytokens.

The agent follows a four-step Observe → Decide → Act → Evaluate loop each cycle. Decision-making is powered by Google's Gemini API (gemini-3.5-flash), with a rule-based fallback that activates when the API is unavailable or rate-limited. A rolling AgentMemory deque (capacity: 100 experiences) tracks action outcomes and prevents the agent from repeating strategies that have failed more than 60% of the time.

System Specifications

Component	Details
Attack Phases	6 (Scanning · Brute Force · Exploit · Persistence · Data Exfiltration · Dormant)
Agent Actions	4 (increase_delay · deploy_fake_vuln · deploy_honeytoken · observe)
Memory Capacity	100 experiences (rolling deque); avoidance threshold: >60% failure rate
AI Model	Google Gemini API — gemini-3.5-flash; rule-based fallback when unavailable
Dashboard	Streamlit — 5 tabs: Dashboard, Cycles, Memory, Deception, Export
Scoring Metrics	Engagement, Risk, Deception, Agent — all 0–100 scale, updated each cycle

2. What the AI Did Well

2.1 Phase-Responsive Decision Making

The agent correctly escalated its response strategy as attackers advanced through threat phases. In observed runs, brute-force activity reliably triggered login delay increases within 1–2 cycles, and exploit-phase behavior prompted fake endpoint deployment in approximately 85% of qualifying cycles. The rule-based fallback — which fired whenever the Gemini API was unavailable — matched the AI's decision quality in straightforward phase scenarios.

2.2 Multi-Attacker Management

The system successfully tracked 4 concurrent simulated attackers across independent state machines during the live demo session, with individual risk scores ranging from 6.7 (scanning phase) to 21.3 (brute-force, high-skill attacker). The agent computed composite phase distributions and selected responses based on the highest-severity active phase rather than averaging — an effective triage strategy.

2.3 Memory-Based Avoidance

AgentMemory successfully prevented repeated failed strategies. With a failure-rate threshold of 60%, actions that performed poorly were suppressed within 3–5 cycles. The memory's rolling 100-experience window kept the agent's decision model current without accumulating stale data from early uninformed cycles.

2.4 Graceful Degradation

When the Gemini API quota was exceeded mid-session (visible in the live event log at 03:12:16), the system fell back to rule-based logic without crashing or stalling. The transition was seamless from a user perspective — the agent continued producing decisions at full speed, with no cycle skipped. This zero-downtime fallback is the most operationally significant design success.

Performance Summary — What Worked

Capability	Result	Evidence
Phase-responsive action selection	~85% correct escalation	Brute force → delay; exploit → fake_vuln within 1–2 cycles
Multi-attacker tracking	4 attackers, 6 phases	Risk scores 6.7–21.3; independent state machines
Memory-based avoidance	Suppression in 3–5 cycles	>60% failure-rate threshold; 100-exp rolling window
Graceful API fallback	0 cycles skipped	Quota exceeded at 03:12:16; rule-based took over instantly
Agent score trend	50 → 75 over 10 cycles	Score chart shows steady climb as successful actions accumulate

3. What the AI Did Poorly

3.1 API Latency

Each Gemini API call introduced 2–3 seconds of latency per decision cycle. In a real network environment where attacker phase transitions can occur in sub-second windows, this response time is operationally unacceptable. During the 10-cycle demo session, Gemini-powered cycles took approximately 2.4 seconds on average versus under 0.05 seconds for rule-based cycles — a 48× speed disadvantage for the AI path.

3.2 Over-Reactivity to Single Events

The agent occasionally switched strategy after a single anomalous phase event before a stable pattern was established. For example, a transient scanning spike from a dormant attacker triggered an 'observe' decision that delayed response to the concurrent brute-force attacker by 1 full cycle. The absence of a minimum-confidence threshold (e.g., require ≥2 consecutive cycles in a phase before escalating) made the agent more reactive than adaptive.

3.3 No Cross-Session Memory

AgentMemory is stored in a Python deque in RAM. Every app restart wipes all 100 stored experiences, resetting the agent to a naive state. In a production deployment with recurring attackers, the agent would relearn the same failure patterns from scratch on each run — negating the memory advantage. No persistence layer (disk, database, or file) was implemented.

3.4 Shallow Context Window for Gemini

The Gemini prompt was capped at the first 1,000 characters of the honeypot status JSON (hardcoded in ask_gemini()). This truncation excluded attacker history, deception artifact details, and score trends — potentially causing the model to make decisions without full context. In one observed session, Gemini recommended 'observe' when the full state showed 2 attackers in persistence phase, a decision the rule-based fallback correctly overrode with 'deploy_honeytoken'.

3.5 Simulated vs. Real Threat Data

Attacker phase transitions are driven by random probability weighted by aggression and skill attributes rather than real network traffic. As a result, the agent was never tested against genuine attack signatures, protocol-level behavior, or evasion techniques. The scoring system's heuristic weights (e.g., risk = base_phase_score × 0.4·skill + 0.3·aggression + 0.3·persistence) are unvalidated against real-world threat intelligence data.

Limitations Summary — What Didn't Work

Limitation	Measured Impact	Root Cause
API latency	2–3 s/cycle (48× slower than rules)	Gemini REST call per cycle; no caching or batching
Over-reactivity	~1 cycle delay in multi-threat scenarios	No minimum-confidence threshold before escalation
No cross-session memory	100% experience loss on restart	In-RAM deque only; no persistence layer
Shallow Gemini context	1,000-char cap; history excluded	Hardcoded truncation in ask_gemini()
Simulated threat data	No real-world validation	Random probability model; no real packet captures or logs

5. Conclusion

The Adaptive Honeypot Controller successfully demonstrated all three core agentic AI requirements: tool use (4 configurable honeypot actions), multi-step reasoning (a complete Observe–Decide–Act–Evaluate loop), and adaptation (memory-based avoidance of failed strategies). The agent managed 4 concurrent attackers across 6 threat phases with an estimated 85% correct phase-to-action mapping and maintained continuous operation through an API quota failure with zero cycle downtime.

The primary weaknesses are architectural rather than algorithmic: API latency makes Gemini unsuitable for real-time defense without a local model fallback, and in-memory-only storage prevents the agent from retaining experience across sessions. These are tractable engineering problems, not fundamental AI limitations — making the system a strong foundation for a production-grade adaptive honeypot.